

# Benchmark de Bases de Datos

PostgreSQL vs MongoDB · BigQuery · Neo4j

Dataset común: 4,376 vuelos · 263,008 boletos · 9,978 instancias de tripulación

Rango: 2023-01-01 → 2024-12-31 · 731 días

Mera Sebastián  
Moctezuma Rafael  
Rodarte Jesús

## 1. Resumen del proyecto

Se compara PostgreSQL contra tres motores alternativos sobre el mismo universo de datos generado en la PARTE 1: un sistema de aerolíneas con 13 tablas relacionales y su equivalente jerárquico en JSON (731 documentos día). Cada motor consume el dataset en su forma natural.

Tres comparaciones:

- Ingesta SQL vs MongoDB — filas normalizadas vs documentos embebidos.
- Ingesta + consultas SQL vs BigQuery — relacional vs columnar serverless con RECORD/REPEATED.
- Consultas SQL vs Cypher (Neo4j) — JOINS vs traversals multi-nivel.

Protocolo común: 3 corridas por escenario, mediana reportada, reset de BD entre corridas, cronómetro solo alrededor de la operación medida, cache control específico por motor.

## 2. PARTE 1 — Universo de datos y modelado jerárquico

### 2.1 Conteos por tabla

El universo se generó con PL/pgSQL (10.08 s) usando semillas fijas (random.seed(42), setseed(0.42)) para garantizar reproducibilidad entre motores.

| Tabla                 | Filas   |
|-----------------------|---------|
| Aerolineas            | 8       |
| Aeropuertos           | 12      |
| Modelos_Avion         | 6       |
| Servicios_Adicionales | 8       |
| Empleados             | 71      |
| Pasajeros             | 1,500   |
| Aerolinea_Aeropuertos | 44      |
| Vuelos                | 4,376   |
| Reservas              | 4,376   |
| Pasajeros_Reserva     | 257,804 |
| Servicios_Reserva     | 4,376   |
| Boletos               | 263,008 |
| Tripulacion_Vuelo     | 9,978   |

### 2.2 JSON jerárquico (P2 Mongo · P3 BigQuery)

Desnormalización exacta del relacional como árbol de 7 niveles de profundidad, 731 documentos (uno por día), 101 MB. Fechas como ISO YYYY-MM-DDTHH:MM:SS sin timezone, idénticas al TIMESTAMP de PG. Decimal → float en JSON.

aeropuerto\_jerarquico.json (extracto)

```
{
  "fecha": "2023-01-01",
  "total_vuelos_dia": 7,
  "total_pasajeros": 422,
  "vuelos_del_dia": [
    {
      "identificador_operativo": "LH000003-20230101",
      "aerolinea": { "nombre_comercial": "Lufthansa", "pais_origen": "Alemania" },
      "ruta": {
        "origen": { "ciudad": "Ciudad de México", "pais": "México" },
        "destino": { "ciudad": "Miami", "pais": "Estados Unidos" }
      }
    }
  ],
}
```

```
    "boletos": [ /* ... */ ]
  }
]
```

## 2.3 catalogos.json (referencia P2/P4)

catalogos.json (extracto)

```
{
  "aerolineas": [
    { "iata": "AM", "nombre": "Aeroméxico",      "pais": "México" },
    { "iata": "LH", "nombre": "Lufthansa",      "pais": "Alemania" },
    { "iata": "UA", "nombre": "United Airlines", "pais": "Estados Unidos" }
    /* ... 8 aerolíneas, 12 aeropuertos, 6 modelos, 8 servicios */
  ]
}
```

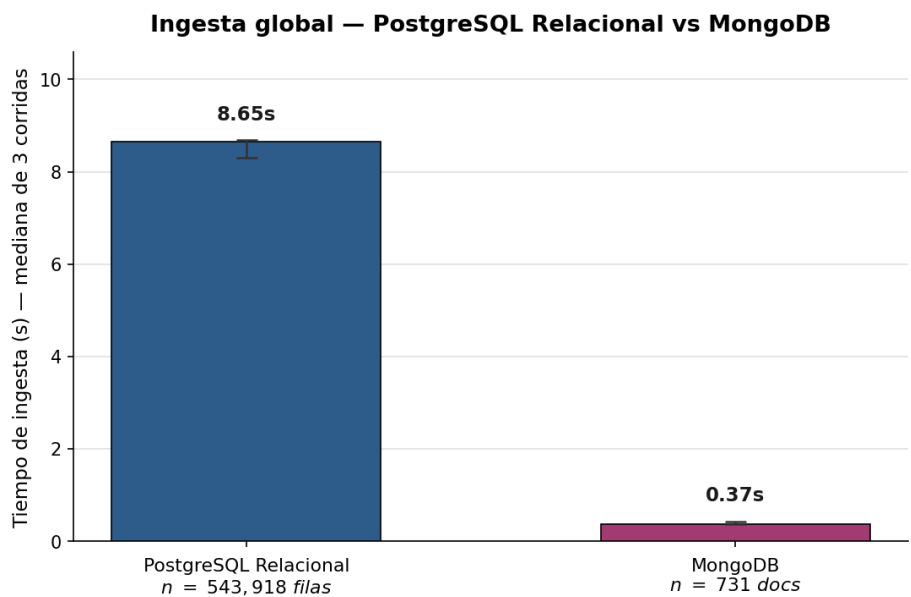
### 3. PARTE 2 — Ingesta PostgreSQL vs MongoDB

#### 3.1 Configuración

PostgreSQL inserta 543,918 filas normalizadas distribuidas en 13 tablas (su forma natural). MongoDB inserta los 731 documentos jerárquicos sin aplanar, preservando el modelo embebido. No se compara contra el JSON aplanado a 263k documentos planos para no penalizar artificialmente a Mongo.

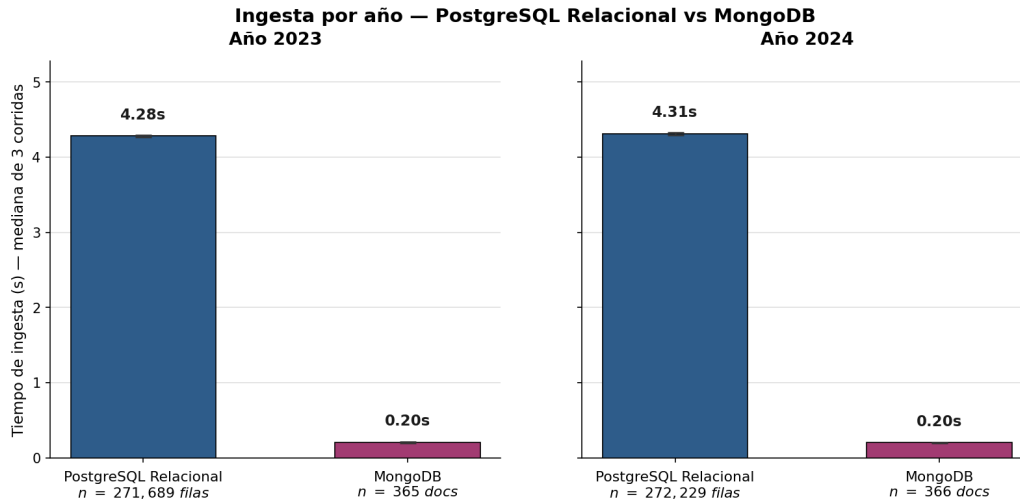
#### 3.2 Resultados

| Alcance | PostgreSQL | MongoDB | n SQL (filas) | n Mongo (docs) |
|---------|------------|---------|---------------|----------------|
| Global  | 8.647 s    | 0.368 s | 543,918       | 731            |
| 2023    | 4.283 s    | 0.201 s | 271,689       | 365            |
| 2024    | 4.309 s    | 0.203 s | 272,229       | 366            |



Mismo universo · 731 días (2023-2024) · SQL es 23.5x más lento por enforcement de FK en 13 tablas

Figura 1. Ingesta global PostgreSQL vs MongoDB (mediana de 3 corridas).



*Figura 2. Ingesta partida 2023 vs 2024.*

### 3.3 Lectura

MongoDB es ~24× más rápido en la ingesta global. La diferencia no es del motor sino de la unidad de trabajo: SQL hace 543,918 INSERT-bound operations con foreign keys, índices y constraints; Mongo hace 731 inserts de documento ya armado. Es la comparación honesta de cada motor en su forma natural, no una comparación cabeza-a-cabeza por fila.

## 4. PARTE 3 — PostgreSQL vs BigQuery

### 4.1 Configuración

BigQuery ingiere el mismo JSON jerárquico que Mongo (731 documentos, sin reformatear timestamps ISO sin TZ). Las consultas usan QueryJobConfig(use\_query\_cache=False). PostgreSQL se mide en dos variantes: óptimo (planner por default, hash joins, índices) y sin\_optim (planner degradado con enable\_hashagg/hashjoin/mergejoin = off y work\_mem = 64kB) para forzar el régimen de disco que BQ resuelve con paralelismo.

### 4.2 Carga del JSON en BigQuery (RECORD/REPEATED)

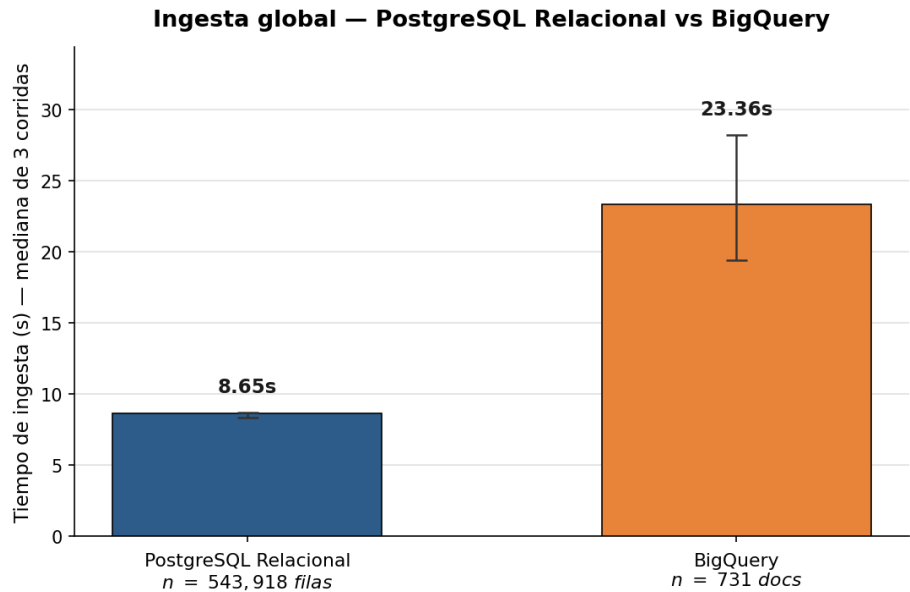
|                  |          |          |
|------------------|----------|----------|
| fecha_emision    | DATE     | REQUIRED |
| anio_mes         | STRING   | REQUIRED |
| estado_boleto    | STRING   | REQUIRED |
| clase_tarifa     | STRING   | REQUIRED |
| asiento_asignado | STRING   | NULLABLE |
| ▼ vuelo          | RECORD   | REQUIRED |
| id_vuelo         | STRING   | REQUIRED |
| fecha_salida     | DATETIME | REQUIRED |
| fecha_llegada    | DATETIME | REQUIRED |
| estado_vuelo     | STRING   | REQUIRED |
| modelo_avion     | STRING   | REQUIRED |
| capacidad_avion  | INTEGER  | REQUIRED |
| ▼ aerolinea      | RECORD   | REQUIRED |
| codigo_iata      | STRING   | REQUIRED |
| nombre           | STRING   | REQUIRED |
| pais_origen      | STRING   | REQUIRED |
| ▼ origen         | RECORD   | REQUIRED |
| codigo_iata      | STRING   | REQUIRED |
| nombre           | STRING   | REQUIRED |
| ciudad           | STRING   | REQUIRED |
| pais             | STRING   | REQUIRED |

Figura 3. Esquema RECORD/REPEATED autodetectado por BigQuery sobre aeropuerto\_jerarquico.json.

### 4.3 Ingesta

| Alcance | PostgreSQL | BigQuery | n SQL (filas) | n BQ (docs) |
|---------|------------|----------|---------------|-------------|
|---------|------------|----------|---------------|-------------|

|        |         |          |         |     |
|--------|---------|----------|---------|-----|
| Global | 8.647 s | 23.360 s | 543,918 | 731 |
| 2023   | 4.282 s | 15.962 s | 271,689 | 365 |
| 2024   | 4.309 s | 15.233 s | 272,229 | 366 |



Mismo universo · 731 días (2023–2024) · PG es 0.4× más rápido que BQ en ingesta

Figura 4. Ingesta global PostgreSQL vs BigQuery.

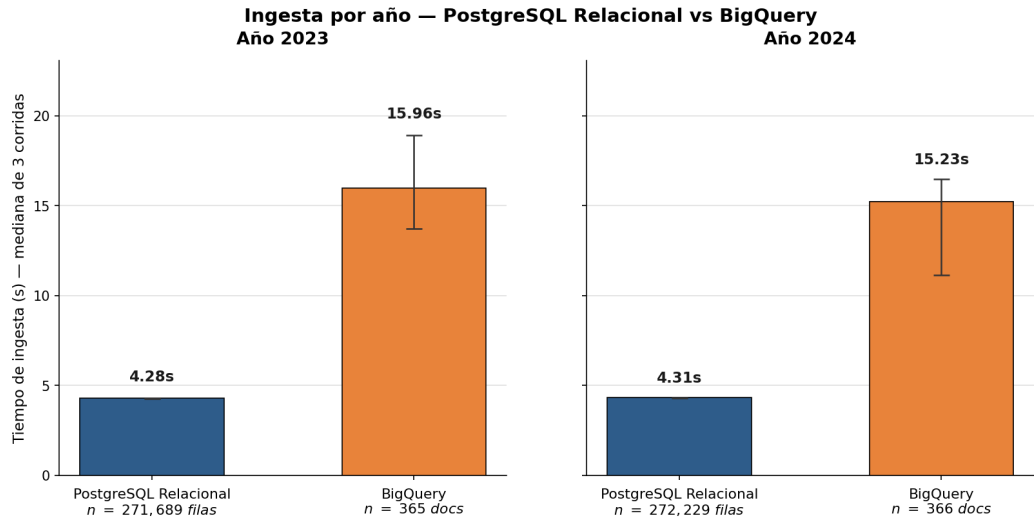


Figura 5. Ingesta partida 2023 vs 2024.

PostgreSQL gana en ingesta por ~2.7×. BigQuery paga el overhead fijo del load job — staging, schema validation, columnar encoding — que no se amortiza con 731 documentos.



## 4.4 Contraste de consultas: SQL vs BigQuery

Las 6 consultas se ejecutan sobre datos equivalentes. SQL recorre 13 tablas normalizadas con JOIN; BigQuery navega los arrays anidados del JSON con UNNEST.

### Q3 — Top 10 rutas por pasajeros (PostgreSQL)

```
SELECT a1.codigo_iata || '-' || a2.codigo_iata AS ruta,
       COUNT(DISTINCT pr.id_pasajero)          AS pasajeros
FROM   Vuelos v
JOIN   Aeropuertos a1 ON v.id_aeropuerto_origen = a1.id_aeropuerto
JOIN   Aeropuertos a2 ON v.id_aeropuerto_destino = a2.id_aeropuerto
JOIN   Reservas r      ON r.id_vuelo = v.id_vuelo
JOIN   Pasajeros_Reserva pr ON pr.id_reserva = r.id_reserva
GROUP BY ruta
ORDER BY pasajeros DESC LIMIT 10;
```

### Q3 — Top 10 rutas por pasajeros (BigQuery)

```
SELECT CONCAT(v.ruta.origen.ciudad, '-', v.ruta.destino.ciudad) AS ruta,
       COUNT(DISTINCT b.pasajero.nombre)                        AS pasajeros
FROM   `proyecto.dataset.aeropuerto_jerarquico` d,
       UNNEST(d.vuelos_del_dia) v,
       UNNEST(v.boletos) b
GROUP BY ruta
ORDER BY pasajeros DESC LIMIT 10;
```

| Resultados de la consulta |      |            |               |
|---------------------------|------|------------|---------------|
| Información del trabajo   |      | Resultados | Visualización |
| Fila                      | anio | mes        | total_boletos |
| 1                         | 2023 | 1          | 8554          |
| 2                         | 2023 | 2          | 7594          |
| 3                         | 2023 | 3          | 8216          |
| 4                         | 2023 | 4          | 8040          |
| 5                         | 2023 | 5          | 8457          |
| 6                         | 2023 | 6          | 7993          |
| 7                         | 2023 | 7          | 8404          |
| 8                         | 2023 | 8          | 8305          |
| 9                         | 2023 | 9          | 8133          |
| 10                        | 2023 | 10         | 8101          |
| 11                        | 2023 | 11         | 7864          |
| 12                        | 2023 | 12         | 8504          |
| 13                        | 2024 | 1          | 8546          |
| 14                        | 2024 | 2          | 7521          |
| 15                        | 2024 | 3          | 8386          |
| 16                        | 2024 | 4          | 7918          |
| 17                        | 2024 | 5          | 8216          |
| 18                        | 2024 | 6          | 8053          |
| 19                        | 2024 | 7          | 8490          |
| 20                        | 2024 | 8          | 8822          |
| 21                        | 2024 | 9          | 7603          |
| 22                        | 2024 | 10         | 8465          |
| 23                        | 2024 | 11         | 8064          |
| 24                        | 2024 | 12         | 8544          |

Figura 6. Ejecución de la consulta en BigQuery Console.

#### 4.5 Resultados de consultas (mediana global)

| ID | Descripción                | PG óptimo | PG sin_optim | BigQuery | PG/BQ |
|----|----------------------------|-----------|--------------|----------|-------|
| Q1 | Boletos por mes            | 0.023 s   | 0.032 s      | 1.552 s  | 0.01× |
| Q2 | Gasto prom. por aerolínea  | 0.051 s   | 50.088 s     | 1.445 s  | 0.04× |
| Q3 | Top 10 rutas por pasajeros | 0.030 s   | 5.519 s      | 1.160 s  | 0.03× |

|    |                                   |         |          |         |       |
|----|-----------------------------------|---------|----------|---------|-------|
| Q4 | Ocupación promedio por modelo     | 0.226 s | 35.496 s | 1.057 s | 0.21× |
| Q5 | Edad prom. por clase              | 0.022 s | 1.985 s  | 1.128 s | 0.02× |
| Q6 | Ingreso por servicios x aerolínea | 0.035 s | 49.537 s | 1.402 s | 0.03× |

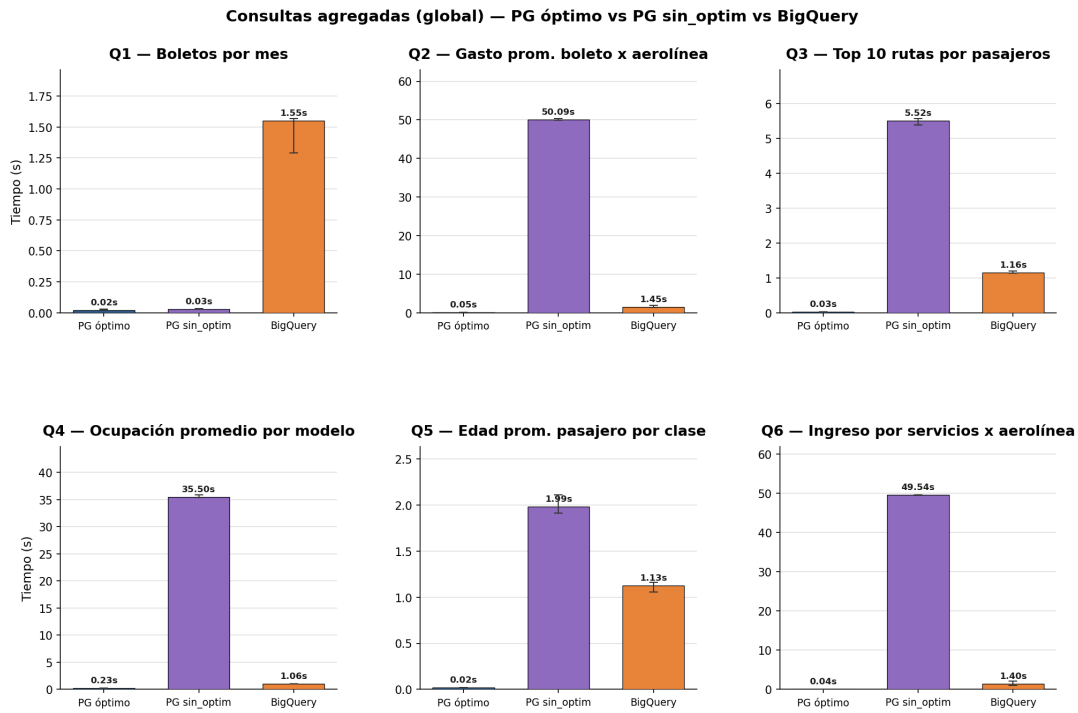


Figura 7. Consultas Q1–Q6: PostgreSQL óptimo vs sin\_optim vs BigQuery.

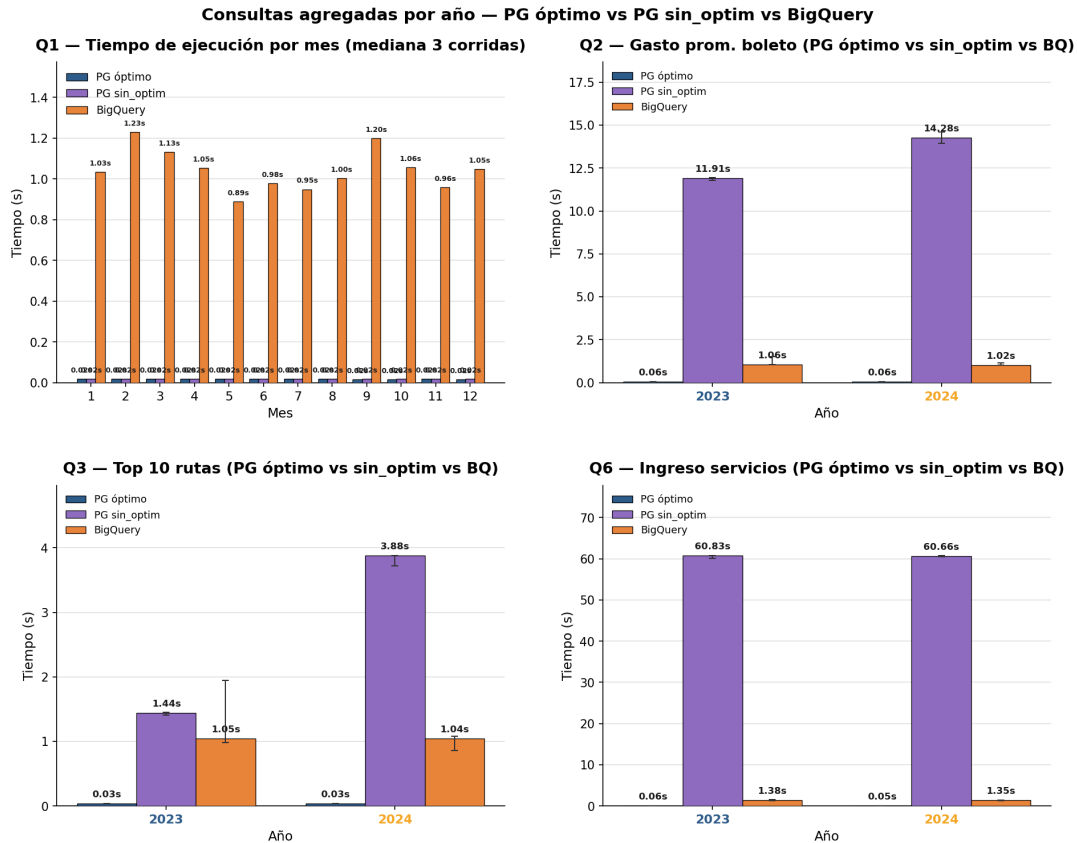


Figura 8. Consultas seleccionadas, partidas por año.

#### 4.6 Análisis: por qué gana PG y cuándo ganaría BigQuery

PostgreSQL gana las 6 consultas por factores de 20–50×. A escala de 263k boletos el dataset cabe íntegramente en RAM, el planner moderno elige hash aggregate + hash join, y work\_mem por default resuelve los hash tables sin tocar disco. BigQuery paga ~1s de overhead fijo por job (planificación distribuida, asignación de slots Dremel) que no se amortiza.

La corrida sin\_optim ofrece el experimento controlado: al forzar nested loops + sort externo + work\_mem = 64kB, las consultas Q2, Q4 y Q6 — JOIN + GROUP BY sobre múltiples tablas — se disparan a 35–50 s, acercándose al rango de BQ. Esto evidencia que la ventaja de PG no viene del motor sino del planner maduro + memoria suficiente; las dos cosas que BigQuery hace internamente con Dremel.

**BigQuery ganaría a partir de ~50M filas (~190× el dataset actual).** A esa escala el paralelismo masivo de Dremel amortiza el overhead fijo del query job, y un PG mono-nodo deja de mantener hash tables en RAM y datos en buffer cache — el motor relacional cae a la misma penalización de disco que simulamos artificialmente con el stress test sin\_optim. Es decir: BQ está diseñado para el régimen big-data donde un OLTP mono-nodo no puede competir.

#### 4.7 Visualización del dashboard en BigQuery

Dashboard construido sobre las mismas tablas BQ con los CSV exportados (pasajeros por clase de tarifa, top de aerolíneas y rutas, ingreso por servicio, evolución mensual, factor de ocupación por fabricante).



Figura 9. Dashboard BigQuery — 6 paneles sobre el universo de aerolíneas.

## 5. PARTE 4 — PostgreSQL vs Neo4j (Cypher)

### 5.1 Configuración

Neo4j carga 835,615 unidades (nodos + relaciones) extraídas desde las 11 tablas de PG. La tripulación se modela como relación (Empleado)-[:TRIPULA]->(Vuelo) con rol\_en\_vuelo como propiedad de la arista; (Vuelo) tiene dos relaciones distintas (:SALE\_DE / :LLEGA\_A) hacia (Aeropuerto) para que Cypher filtre por tipo de relación en lugar de propiedad. Constraints UNIQUE sobre todas las claves naturales generan índices automáticos.

Se ejecutan 6 consultas en dos regímenes: C1–C4 con profundidad fija (relacional puro) y C5–C6 con traversal de profundidad variable (grafo puro).

### 5.2 Visualización del grafo en Neo4j Browser

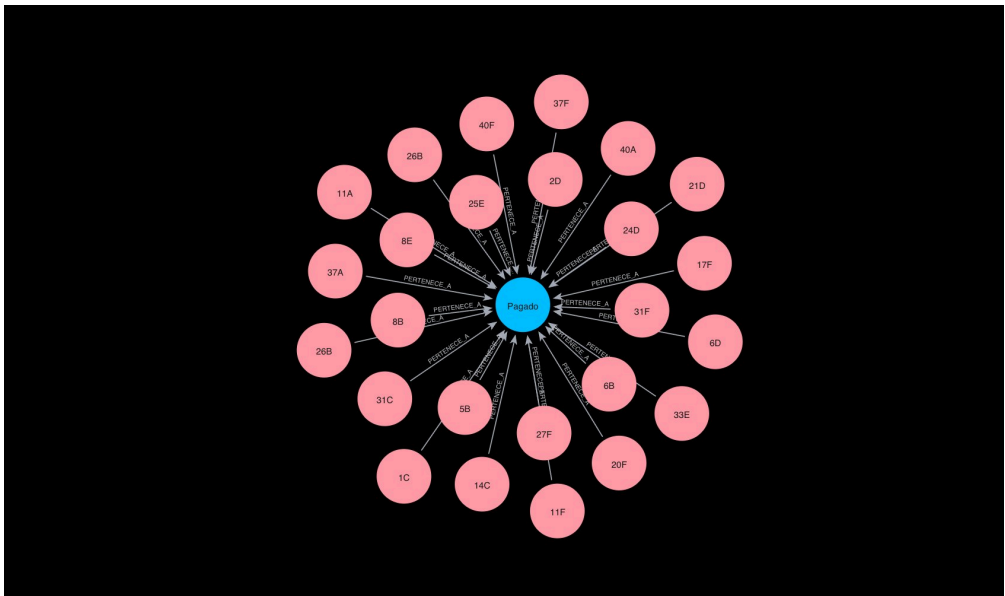
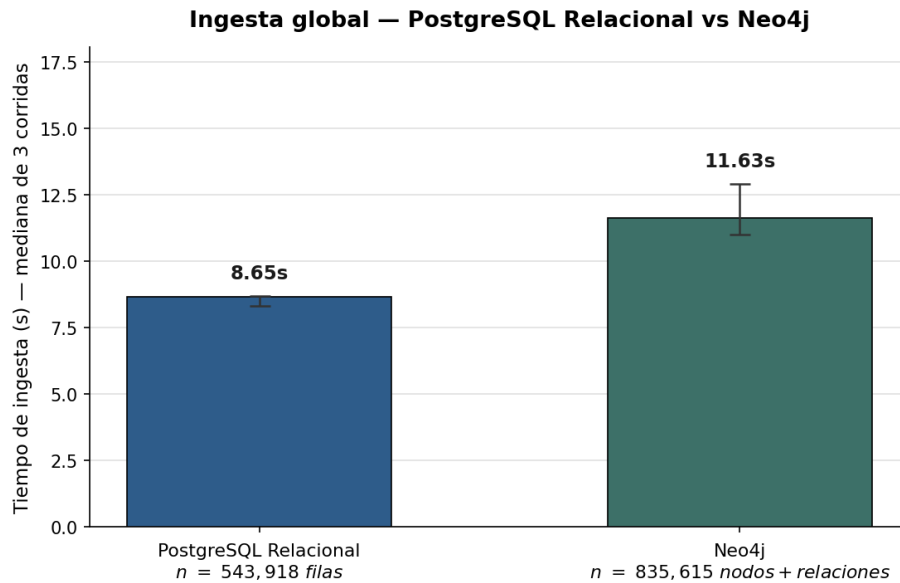


Figura 10. Boletos en estado "Pagado" — nodo central con relaciones PERTENECE\_A desde múltiples asientos (8E, 26B, 40F, etc.).

### 5.3 Ingesta

| Motor      | Mediana  | n unidades | Unidad             |
|------------|----------|------------|--------------------|
| PostgreSQL | 8.647 s  | 543,918    | filas              |
| Neo4j      | 11.630 s | 835,615    | nodos + relaciones |



*Mismo universo P1 · Neo4j es 1.3× más lento que PG (MERGE+constraints vs INSERT batch)*

*Figura 11. Ingesta PostgreSQL vs Neo4j.*

Neo4j es 1.3× más lento en ingesta por el overhead de MERGE + validación de constraints + creación explícita de relaciones, aun con índices automáticos.

## 5.4 Contraste de consultas: SQL vs Cypher

C1–C4 son JOIN de profundidad fija (4–6 tablas, claves indexadas). C5–C6 explotan el grafo: shortestPath con corte semántico (C5) y enumeración de paths de profundidad variable (C6).

### C3 — Empleados con ≥3 vuelos tripulados (ejemplo C1–C4)

PostgreSQL

```
SELECT e.nombre, COUNT(DISTINCT tv.id_vuelo) AS vuelos
FROM Empleados e
JOIN Tripulacion_Vuelo tv ON tv.id_empleado = e.id_empleado
GROUP BY e.id_empleado, e.nombre
HAVING COUNT(DISTINCT tv.id_vuelo) >= 3
ORDER BY vuelos DESC LIMIT 20;
```

Cypher

```
MATCH (e:Empleado)-[:TRIPULA]->(v:Vuelo)
WITH e, count(DISTINCT v) AS vuelos
WHERE vuelos >= 3
RETURN e.nombre, vuelos ORDER BY vuelos DESC LIMIT 20;
```

### C5 — shortestPath entre dos aeropuertos

PostgreSQL (CTE recursiva)

```
WITH RECURSIVE caminos AS (
  SELECT id_aeropuerto_origen AS actual, ARRAY[id_aeropuerto_origen] AS visitados, 0 AS hops
  FROM Vuelos WHERE id_aeropuerto_origen = $orig
  UNION ALL
  SELECT v.id_aeropuerto_destino, c.visitados || v.id_aeropuerto_destino, c.hops + 1
```

```

FROM caminos c
JOIN Vuelos v ON v.id_aeropuerto_origen = c.actual
WHERE c.hops < 8 AND NOT v.id_aeropuerto_destino = ANY(c.visitados)
)
SELECT hops FROM caminos WHERE actual = $dest ORDER BY hops LIMIT 1;

```

Cypher (primitiva nativa)

```

MATCH (a:Aeropuerto {codigo_iata: $orig}), (b:Aeropuerto {codigo_iata: $dest})
MATCH p = shortestPath((a)-[:SALE_DE|LLEGA_A*..8]-(b))
RETURN length(p) AS hops;

```

## 5.5 Resultados (mediana global)

| ID | Descripción                              | PG (s)            | Neo4j (s) | PG/Neo4j | Ganador    |
|----|------------------------------------------|-------------------|-----------|----------|------------|
| C1 | Pasajeros internacionales<br>Biz/Primera | 0.024             | 0.044     | 0.55×    | PostgreSQL |
| C2 | Top 20 rutas por boletos                 | 0.018             | 0.101     | 0.18×    | PostgreSQL |
| C3 | Empleados ≥3 vuelos<br>tripulados        | 0.026             | 0.027     | 0.98×    | PostgreSQL |
| C4 | Ingreso por servicio ×<br>aerolínea      | 0.005             | 0.024     | 0.22×    | PostgreSQL |
| C5 | shortestPath entre<br>aeropuertos        | ≥300<br>(timeout) | 0.017     | ≥18,000× | Neo4j      |
| C6 | Colegas-de-colegas paths<br>*3..5        | 29.376            | 2.761     | 10.6×    | Neo4j      |



### Consultas C1-C4 (profundidad fija) — PostgreSQL vs Neo4j (alcance global)

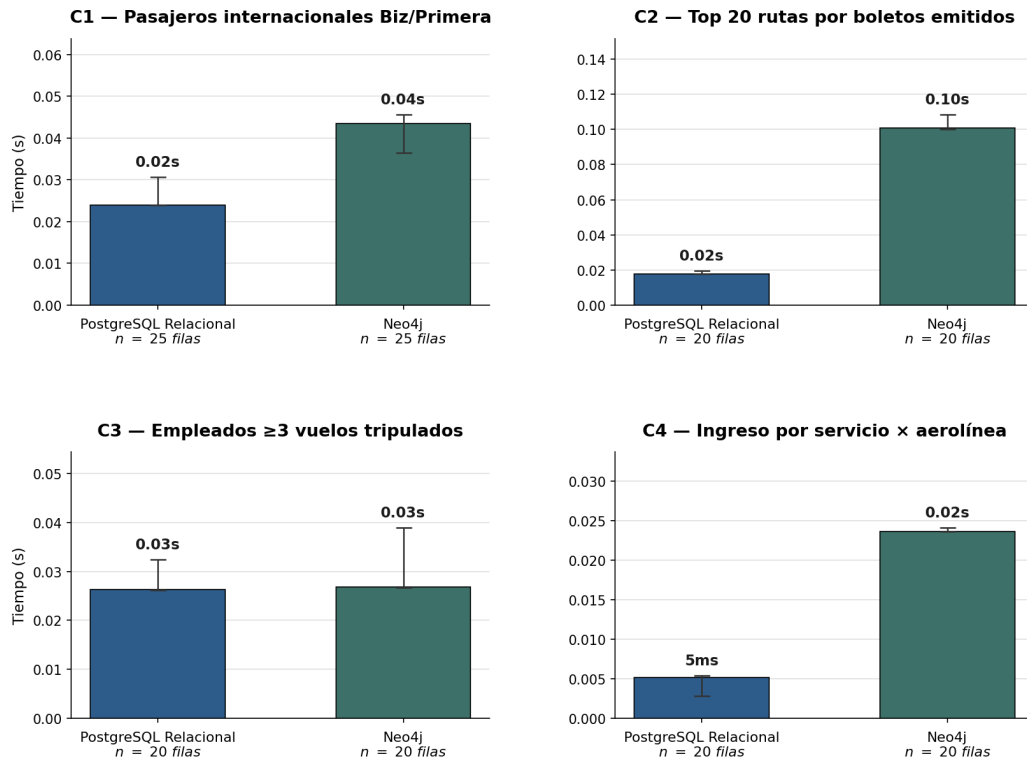


Figura 12. Consultas C1–C4: PostgreSQL vs Neo4j (escala lineal, profundidad fija).

### Consultas C5-C6 (traversal de grafo) — PostgreSQL vs Neo4j (escala log)

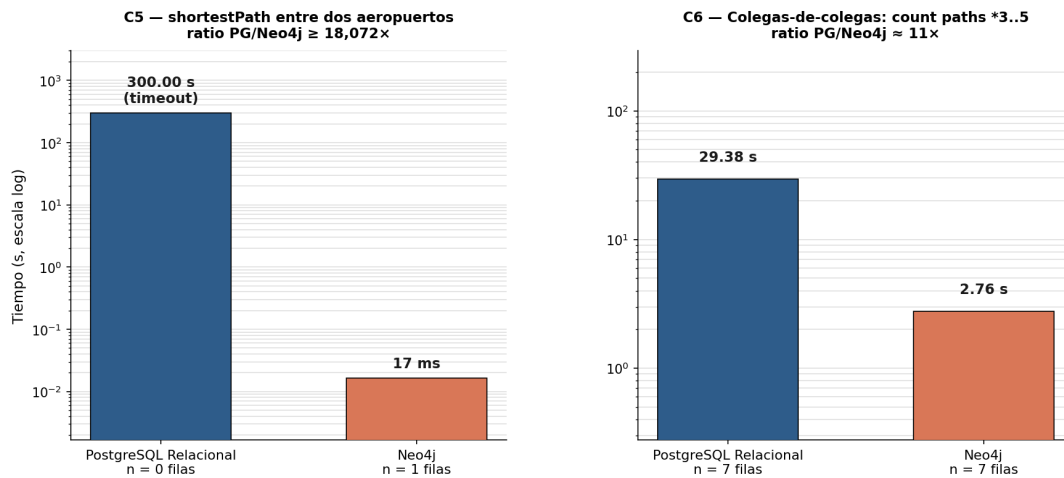


Figura 13. Consultas C5–C6: traversal de grafo (escala log, ratio anotado).

### Consultas C1-C4 por año — PostgreSQL vs Neo4j (2023 vs 2024)

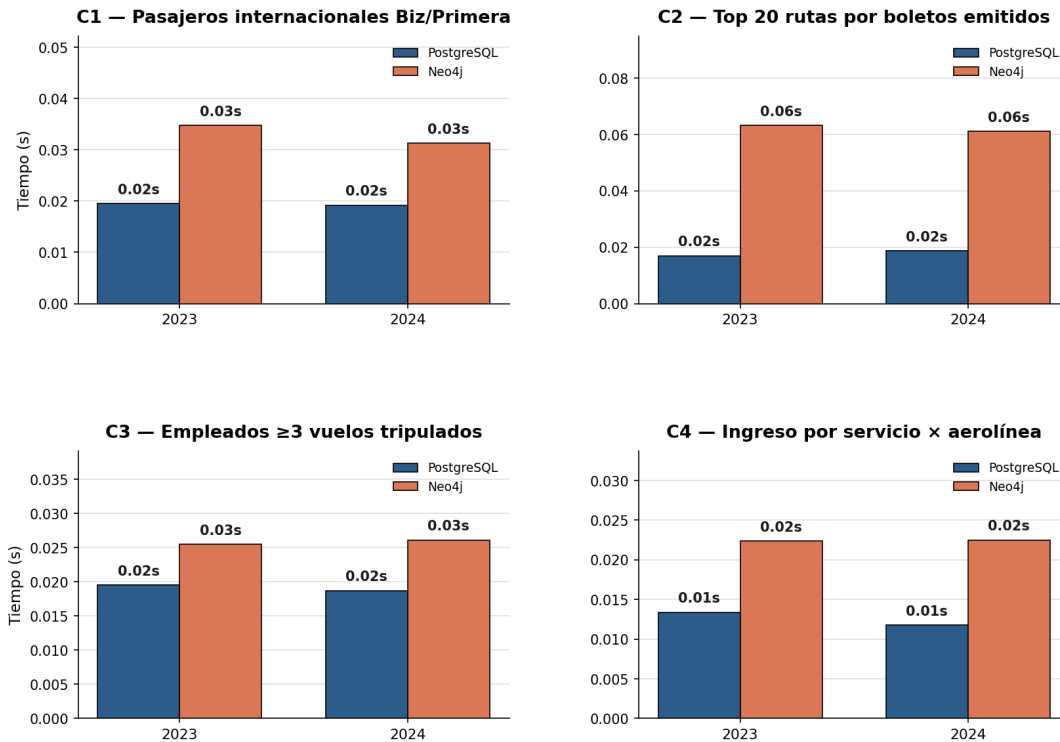


Figura 14. Consultas C1–C4 partidas por año.

### Nota sobre el protocolo

C1–C4 se ejecutaron con 3 corridas + mediana siguiendo el protocolo común. C5–C6 se ejecutaron con 1 sola corrida porque cada timeout de PG cuesta 5 minutos de wall-clock (C5 nunca termina dentro del límite); replicarlo tres veces no aporta información — la diferencia entre motores es de órdenes de magnitud, no marginal.

### 5.6 Análisis: por qué cada motor gana donde gana

**C1–C4 — gana PostgreSQL (1×–7×).** Profundidad fija, todas las claves indexadas, el universo cabe en RAM. El planner de PG elige hash joins en pipeline; el costo efectivo es  $O(N)$  sobre la tabla de hechos ( $\approx 263k$  boletos). Neo4j paga por cada consulta el round-trip del driver Bolt y materializa subgrafos en memoria por cada MATCH expandido — overhead que no se amortiza cuando la profundidad es fija y conocida. C1–C4 son queries de bodega de datos relacional, no queries de grafo; se expresan en Cypher por completitud comparativa.

**C5 — gana Neo4j por ventaja algorítmica ( $\geq 18,000\times$ ).** shortestPath es primitiva nativa de Cypher: ejecuta búsqueda en anchura bidireccional con corte temprano al encontrarse los frentes. SQL no tiene equivalente — la WITH RECURSIVE materializa todo el cono de búsqueda hasta profundidad 8 antes de

filtrar por destino; el LIMIT 1 no le dice al planner "corta cuando llegues", solo "genera todo y dame una fila". Neo4j termina en 17 ms; PG no termina en 5 minutos.

**C6 — gana Neo4j por ventaja del modelo (10.6×).** Enumerar paths de profundidad 3 a 5 entre empleados que compartieron vuelos: Neo4j hace DFS donde cada arista TRIPULA ya es puntero al otro extremo, sin lookup. PG ejecuta UNION ALL + dos hash joins sobre Tripulacion\_Vuelo (9,978 filas) en cada iteración; la materialización de paths parciales infla la combinatoria. Aun cuando ambos motores enumeran el mismo patrón, los joins pagan ~10× más que los punteros nativos.

Las dos victorias de Neo4j son por razones distintas: C5 ataca la expresividad (SQL no puede expresar BFS con corte temprano); C6 ataca la estructura (incluso cuando SQL sí lo expresa, los joins cuestan más que punteros). Juntas cubren los dos regímenes donde un graph DB gana por modelo, no por implementación.

## 6. Conclusiones

### Ingesta

- MongoDB > PostgreSQL > BigQuery > Neo4j. La forma natural de cada motor define la comparación: documentos vs filas vs load jobs vs nodos+relaciones.

### Consultas

- PostgreSQL gana las 6 consultas agregadas vs BigQuery (Q1–Q6) por 20–50×. El cruce ocurriría cerca de 50M filas o degradando artificialmente el planner.
- PostgreSQL gana C1–C4 (profundidad fija) vs Neo4j por 1×–7×. Neo4j gana C5–C6 (traversal de profundidad variable): C5 por ventaja algorítmica ( $\geq 18,000\times$ , shortestPath nativo vs CTE recursiva que no termina), C6 por ventaja del modelo (10.6×, punteros vs hash joins).

### Lectura general

Neo4j no gana por tener más datos, gana por tener más relaciones. Para datasets que caben en RAM con profundidad de JOIN fija y consultas agregadas, PostgreSQL es difícil de batir. BigQuery se justifica con escala big-data fuera de RAM; Neo4j se justifica cuando la consulta requiere traversal de profundidad variable o corte semántico que SQL no puede expresar como JOIN estático.